

1 General WinBUGS principles

- WinBUGS can be downloaded from [here](#). Instructions on how to download are on the home page
- The WinBUGS user manual can be found [here](#)
- WinBUGS has its own syntax. For the most part it is similar to R
- You will need to define your model, data set, and initial values.
 - These can all be defined in the same file, or in different files. Whatever you prefer.
 - Data must be list format or rectangular, but data type does not need to be consistent if using multiple data sets.
 - Initial conditions are optional and can be generated by WinBUGS if need be, but it's generally better to specify if you can

2 Model set up

1. Define your model, data, and initial values in a new window (File;New).
2. Check the model for syntax errors: In WinBUGS, open model ; specification. In the file where you defined your model, highlight the word "model" then click on "check model" in the model specification window. You'll might get a message that says "the new model will replace the old one". This is fine, click okay and continue. Any errors will be displayed somewhat cryptically on the bottom left of the screen. If everything is okay, you'll get the message "model is syntactically correct"
3. Load your data: If your data is in a list, highlight the word "list" in the list that you used to define your data and click on "load data" in the model specification window. If your data is a rectangular array highlight the first line and click on "load data". If your data is in more than one list/ array, repeat as many times as you need to to load all of your data
4. Compile: Once you've loaded your model and data, click "compile". If you want to use BGR diagnosis later, increase the number of chains from 1 to any other postive integer. I would recommend a small one unless you really like clicking things.
5. Load your initial conditions: highlight the word "list" in your list of initial conditions, then click "load inits" in the model specification window. If you have multiple chains, you need to load initial conditions for all of them. If you are planning to use Gelman-Rubin diagnostics later, you should define unique and wide spread initial conditions for each chain. If you don't have initial values defined for all of your distribution parameters, click on "gen inits"
6. Generate some throw away values to help ensure the model converges: open Model ; Update. The default number of iterations is 1000 – change this if you want to do more (or less) throw away iterations, then click "update"
7. Set variables you wish to make inferences about: open Inference ; Samples and enter each variable you want to make inferences about in the window next to the word "node". After each variable, click "set" before entering the next one
8. Make sure the model converges: with your variables set, run some more updates. Check the *history* and *bgr* metrics to see if your model has converged

[width=0.5]figures/WinBUGS_code.png

Figure 1: WinBUGS sample code

9. Produce posterior distributions: Clear and reset your variables of interest in the Inference Samples window. Return to the Model Update window. Enter the desired number of iterations and click "update"

3 Analysis

Now that you have generated your posterior distributions, you can look at a summary of these distributions. Go back to the Inference Samples window and type a "*" . This indicates that you want to look at all variables being tracked. The most important tools for our purposes are summarized below. Information about all possible buttons you could press can be found in the WinBUGS user manual

- stats: this will give a summary of the estimated statistics of the posterior distributions for each variable, pooling information from all selected chains. Note that the more parameters you're estimating, the lower the accuracy of the estimates will be
- density: this will give you a visual of the smoothed kernel density (i.e. distribution) estimate for each variable
- history: look at the complete trace (variable value against iteration number) for each variable. This is where you're looking for "fuzzy caterpillar" vs "snake"
- bgr diag: calculates and plots the Gelman-Rubin convergence statistic. The BGR statistic is plotted by the red line, and you want this to be about 1 (± 0.1)

4 Example

Consider a PPK model

$$y_{ij} = f(\phi_i, x_{ij}) + \epsilon_{ij}$$

where $y_i \sim N(f(\phi_{ij}, x_{ij}), \sigma^2)$ and $\phi_i \sim N(\theta, \Omega)$ for $i = 1, \dots, N$ patients. Note that for simplicity we are considering the same function f for each patient, and constant parameters for each individual. Consider a simple one compartment model

$$f(\phi_i, x) = \frac{Dose_i}{V_i} \exp\left(-\frac{CL_i}{V_i} x\right)$$

Dose is known, so $\theta = [V_i, CL_i]$ Suppose we want Bayesian estimates of population clearance, population volume of distribution, patient variability, and uncertainty. Sample code for this scenario is presented below. You can now do steps 1-5. Note that initial conditions are not provided for ϕ , so we will have to prompt WinBUGS to generate these from the other given model information (using "gen inits") after we load the initial conditions that we do have. We'll use 1000 throw away updates

4.0.1 Convergence

The variables we wish to make inferences about are V , CL , σ , and Ω . Set these, then run some more updates to check for convergence. We will try 4000 iterations. We will only look at results for σ for sake of space, but all plots should look similar. First we check history for the "fuzzy caterpillar" Looks good for both chains. Now we'll check bgr for Gelman-Rubin convergence The green and blue lines overlap, and the red line is approximately 1. This is what we want, so we're good to move on.

[width=0.5]figures/history.png

Figure 2: Trace History

[width=0.5]figures/bgr.png

Figure 3: bgr plot

4.0.2 Analysis

Clear the variables and run the model for another 10000 updates. Recall that as an additional check, we want MC error to be $< 5\%$ of the standard deviation. We have

$$\frac{\text{MC error}}{sd} \approx 0.033 < 0.05$$

so this condition is satisfied as well

[width=0.5]figures/stats.png

Figure 4: Model statistics for σ

[width=0.5]figures/density.png

Figure 5: Predicted density of σ